Ex. No: 01	Data Definition Language (DDL) commands in RDBMS

AIM:

ALGORITHM:

Step 1: Start
Step 2: Login to Oracle SQL
Step 3: Create a table
Step 4: Insert records
Step 5: Perform ALTER, TRUNCATE, RENAME, and DROP operations
Step 6: Observe the output
Step 7: Stop

DDL (DATA DEFINITION LANGUAGE)

Data Definition Language (DDL) statements are used to define the database structure or schema. DDL having the following commands

- **CREATE** - To create objects in the database
- **ALTER** - Alters the structure of the database
- **DROP** - Delete objects from the database
- **TRUNCATE** - Remove all records from a table, including all spaces allocated for the records are removed
- **RENAME** - Rename an object

1. CREATE TABLE:

- **Creating a new table**
  Create Table command is used in creation of table(s). It creates an empty table has the following form:

  Syntax:
  CREATE TABLE table_name (
  column1_name data_type
  [constraints], column2_name
  data_type [constraints],
  ...
  columnN_name data_type [constraints]);

```
mysql> CREATE TABLE Department (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50),
    ->     hod_name VARCHAR(50),
    ->     total_staff INT,
    ->     total_students INT,
    ->     building VARCHAR(50),
    ->     established_year INT
    -> );
Query OK, 0 rows affected (0.362 sec)
```

2. ALTER TABLE:

```
mysql> INSERT INTO Department VALUES
    -> (101, 'Computer Science', 'Dr. Kumar', 25, 600, 'Block A', 2005);
Query OK, 1 row affected (0.092 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (102, 'Information Technology', 'Dr. Priya', 20, 450, 'Block B', 2008);
Query OK, 1 row affected (0.049 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (103, 'Electronics and Communication', 'Dr. Ravi', 18, 400, 'Block C', 2003);
Query OK, 1 row affected (0.034 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (104, 'Mechanical Engineering', 'Dr. Suresh', 30, 700, 'Block D', 2000);
Query OK, 1 row affected (0.039 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (105, 'Civil Engineering', 'Dr. Meena', 15, 350, 'Block E', 2006);
Query OK, 1 row affected (0.069 sec)
```

Syntax:
ALTER TABLE table_name
[{ADD | MODIFY | DROP} column_name
column_definition][CONSTRAINT constraint_name]
[{ADD | DROP} {PRIMARY KEY | FOREIGN KEY (column_name) REFERENCES table_name (column_name)}]

[{ENABLE | DISABLE} TRIGGER trigger_name]);

To add a column:

Syntax:

ALTER TABLE table_name ADD column_name data_type [constraints];
Description: To add a column into a database.

SQL> ALTER TABLE Transaction ADD Branch_Name VARCHAR2(20);
Output: Table altered.

To change the datatype of a column:

Syntax:-ALTER TABLE table_name MODIFY column_name datatype;
Description: To change the datatype of a column.

SQL> ALTER TABLE Transaction MODIFY Customer_Name VARCHAR2(40);

To delete a column:

Syntax: ALTER TABLE table_name DROP column column_name;

Description: To delete a column.

SQL> ALTER TABLE Transaction DROP column Branch_Name;

```
mysql> SELECT * FROM Department;
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
| dept_id | dept_name                      | hod_name   | total_staff | total_students | building | established_year |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
|     101 | Computer Science               | Dr. Kumar  |          25 |            600 | Block A  |             2005 |
|     102 | Information Technology         | Dr. Priya  |          20 |            450 | Block B  |             2008 |
|     103 | Electronics and Communication  | Dr. Ravi   |          18 |            400 | Block C  |             2003 |
|     104 | Mechanical Engineering         | Dr. Suresh |          30 |            700 | Block D  |             2000 |
|     105 | Civil Engineering              | Dr. Meena  |          15 |            350 | Block E  |             2006 |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
5 rows in set (0.010 sec)

mysql>  ALTER TABLE Department ADD COLUMN location VARCHAR(50);
Query OK, 0 rows affected (0.285 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Department;
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+----------+
| dept_id | dept_name                      | hod_name   | total_staff | total_students | building | established_year | location |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+----------+
|     101 | Computer Science               | Dr. Kumar  |          25 |            600 | Block A  |             2005 | NULL     |
|     102 | Information Technology         | Dr. Priya  |          20 |            450 | Block B  |             2008 | NULL     |
|     103 | Electronics and Communication  | Dr. Ravi   |          18 |            400 | Block C  |             2003 | NULL     |
|     104 | Mechanical Engineering         | Dr. Suresh |          30 |            700 | Block D  |             2000 | NULL     |
|     105 | Civil Engineering              | Dr. Meena  |          15 |            350 | Block E  |             2006 | NULL     |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+----------+
5 rows in set (0.011 sec)
```

```
mysql> ALTER TABLE Department DROP COLUMN location;
Query OK, 0 rows affected (0.173 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>  SELECT * FROM Department;
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
| dept_id | dept_name                      | hod_name   | total_staff | total_students | building | established_year |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
|     101 | Computer Science               | Dr. Kumar  |          25 |            600 | Block A  |             2005 |
|     102 | Information Technology         | Dr. Priya  |          20 |            450 | Block B  |             2008 |
|     103 | Electronics and Communication  | Dr. Ravi   |          18 |            400 | Block C  |             2003 |
|     104 | Mechanical Engineering         | Dr. Suresh |          30 |            700 | Block D  |             2000 |
|     105 | Civil Engineering              | Dr. Meena  |          15 |            350 | Block E  |             2006 |
+---------+--------------------------------+------------+-------------+----------------+----------+------------------+
5 rows in set (0.010 sec)
```

3. TRUNCATE TABLE

Syntax: TRUNCATE TABle table_name;

Description: To delete the data inside the table and not the table itself.

SQL> TRUNCATE TABLE Transaction;

4. DROP TABLE:

Syntax: DROP TABLE table_name;

Description: To delete the entire table. **SQL>** DROP TABLE Transaction_Details;

5. RENAME A TABLE

Rename command is used to give new names for existing tables. SQL> **RENAME Transaction TO Transaction_Details;**

```
mysql> RENAME TABLE Department TO Dept;
Query OK, 0 rows affected (0.305 sec)

mysql>  DROP TABLE Dept;
Query OK, 0 rows affected (0.247 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

Ex.No:02	Data Manipulation Language (DML), Data Control Language (DCL) commands in RDBMS

AIM:

ALGORITHM:

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the username and password.
Step 4**:** Type the commands to manipulate the table and perform operations.
Step 5: The output is displayed.
Step 6: Stop.

TABLE CREATION:

1. INSERT INTO:

To INSERT values in all field

Syntax: INSERT into table_name VALUES (value1,values 2,..);

Description: To insert new data into database.

To INSERT values in particular field

Syntax: INSERT into table_name (field names) values (value 1,…);

Description: To insert values in particular field.

To INSERT values dynamically

Syntax: INSERT into table_name VALUES (&field name1,& field name2);
Description: To INSERT data into database dynamically.

```
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Department (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50),
    ->     hod_name VARCHAR(50),
    ->     total_staff INT,
    ->     total_students INT,
    ->     building VARCHAR(50),
    ->     established_year INT
    -> );
```

```
mysql> INSERT INTO Department VALUES
    -> (101, 'Computer Science', 'Dr. Kumar', 25, 600, 'Block A', 2005),
    -> (102, 'Information Technology', 'Dr. Priya', 20, 450, 'Block B', 2008),
    -> (103, 'Electronics and Communication', 'Dr. Ravi', 18, 400, 'Block C', 2003),
    -> (104, 'Mechanical Engineering', 'Dr. Suresh', 30, 700, 'Block D', 2000),
    -> (105, 'Civil Engineering', 'Dr. Meena', 15, 350, 'Block E', 2006);
Query OK, 5 rows affected (0.096 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Department;
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
| dept_id | dept_name                     | hod_name  | total_staff | total_students | building | established_year |
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
|     101 | Computer Science              | Dr. Kumar |          25 |           600 | Block A  |             2005 |
|     102 | Information Technology        | Dr. Priya |          20 |           450 | Block B  |             2008 |
|     103 | Electronics and Communication | Dr. Ravi  |          18 |           400 | Block C  |             2003 |
|     104 | Mechanical Engineering        | Dr. Suresh |         30 |           700 | Block D  |             2000 |
|     105 | Civil Engineering             | Dr. Meena |          15 |           350 | Block E  |             2006 |
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
5 rows in set (0.008 sec)
```

2. UPDATE:

Syntax: UPDATE TABLE_name SET column_name=value WHERE some column=somevalue;

Description: To update data in a database.

```
mysql> UPDATE Department
    -> SET total_students = 800
    -> WHERE dept_id = 103;
Query OK, 1 row affected (0.136 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Department;
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
| dept_id | dept_name                     | hod_name  | total_staff | total_students | building | established_year |
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
|     101 | Computer Science              | Dr. Kumar |          25 |           600 | Block A  |             2005 |
|     102 | Information Technology        | Dr. Priya |          20 |           450 | Block B  |             2008 |
|     103 | Electronics and Communication | Dr. Ravi  |          18 |           800 | Block C  |             2003 |
|     104 | Mechanical Engineering        | Dr. Suresh |         30 |           700 | Block D  |             2000 |
|     105 | Civil Engineering             | Dr. Meena |          15 |           350 | Block E  |             2006 |
+---------+-------------------------------+-----------+-------------+---------------+----------+------------------+
5 rows in set (0.011 sec)
```

3. DELETE:

Syntax: DELETE FROM table_name WHERE column_name=value;

Description: To delete data from database.

4. SELECT

Syntax: SELECT *FROM table_name;

Description: To extract data from a database.

```
mysql> UPDATE Department
    -> SET total_students = 800
    -> WHERE dept_id = 103;
Query OK, 1 row affected (0.136 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Department;
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
| dept_id | dept_name                    | hod_name  | total_staff | total_students | building | established_year |
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
|     101 | Computer Science             | Dr. Kumar |         25 |           600 | Block A  |            2005 |
|     102 | Information Technology       | Dr. Priya |         20 |           450 | Block B  |            2008 |
|     103 | Electronics and Communication | Dr. Ravi  |         18 |           800 | Block C  |            2003 |
|     104 | Mechanical Engineering       | Dr. Suresh |         30 |           700 | Block D  |            2000 |
|     105 | Civil Engineering            | Dr. Meena |         15 |           350 | Block E  |            2006 |
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
5 rows in set (0.011 sec)

mysql> SELECT dept_id, total_students FROM Department;
+---------+---------------+
| dept_id | total_students |
+---------+---------------+
|     101 |           600 |
|     102 |           450 |
|     103 |           800 |
|     104 |           700 |
|     105 |           350 |
+---------+---------------+
5 rows in set (0.013 sec)

mysql> SELECT * FROM Department
    -> WHERE total_students > 500;
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
| dept_id | dept_name                    | hod_name  | total_staff | total_students | building | established_year |
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
|     101 | Computer Science             | Dr. Kumar |         25 |           600 | Block A  |            2005 |
|     103 | Electronics and Communication | Dr. Ravi  |         18 |           800 | Block C  |            2003 |
|     104 | Mechanical Engineering       | Dr. Suresh |         30 |           700 | Block D  |            2000 |
+---------+------------------------------+-----------+------------+---------------+----------+-----------------+
3 rows in set (0.010 sec)
```

DCL commands are used to control access and permissions in the database.

1. GRANT:

Syntax:

GRANT privilege_name
ON table_name
TO user_name;

2. REVOKE:

Syntax:

REVOKE privilege_name
ON table_name
FROM user_name;

```
mysql> CREATE USER 'user1'@'localhost'
    -> IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.162 sec)

mysql> GRANT ALL PRIVILEGES
    -> ON CollegeDB.Department
    -> TO 'user1'@'localhost';
Query OK, 0 rows affected (0.072 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.057 sec)

mysql> REVOKE SELECT ON CollegeDB.Department
    -> FROM 'user1'@'localhost';
Query OK, 0 rows affected (0.081 sec)

mysql> REVOKE INSERT, UPDATE ON CollegeDB.Department
    -> FROM 'user1'@'localhost';
Query OK, 0 rows affected (0.083 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

Ex.No:03	IMPLEMENTATION OF SUB QUERIES

AIM:

ALGORITHM:

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the username and password.
Step 4**:** Type the commands to manipulate the table and perform operations.
Step 5: The output is displayed.
Step 6: Stop.

DESCRIPTION

A Subquery is a query within another SQL query and embedded within the WHERE clause.

- A subquery can be placed in a number of SQL clauses like WHERE clause, FROMclause, HAVING clause.
- You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements alongwith
- the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery is a query within another query. The outer query is known as the main query,
- and the inner query is known as a subquery.
- Subqueries are on the right side of the comparison operator.
- A subquery is enclosed in parentheses.
- In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

Subqueries with the Select Statement

SQL subqueries are most frequently used with the Select statement

Syntax:

```
SELECT
column_nameFROM table_name
WHERE column_name expression operator
( SELECT column_name  from table_name WHERE ... ):
```

```
mysql> INSERT INTO Department VALUES (101, 'Computer Science', 'Dr. Kumar', 25, 600, 'Block A', 2005);
Query OK, 1 row affected (0.072 sec)

mysql>
mysql> INSERT INTO Department VALUES (102, 'Information Technology', 'Dr. Priya', 20, 450, 'Block B', 2008);
Query OK, 1 row affected (0.039 sec)

mysql>
mysql> INSERT INTO Department VALUES (103, 'Electronics and Communication', 'Dr. Ravi', 18, 400, 'Block C', 2003);
Query OK, 1 row affected (0.050 sec)

mysql>
mysql> INSERT INTO Department VALUES (104, 'Mechanical Engineering', 'Dr. Suresh', 30, 700, 'Block D', 2000);
Query OK, 1 row affected (0.037 sec)

mysql>
mysql> INSERT INTO Staff VALUES (1, 101, 'Arun', 'Professor');
Query OK, 1 row affected (0.053 sec)

mysql>
mysql> INSERT INTO Staff VALUES (2, 102, 'Divya', 'Assistant Professor');
Query OK, 1 row affected (0.053 sec)

mysql>
mysql> INSERT INTO Staff VALUES (3, 103, 'Karthik', 'Lab Assistant');
Query OK, 1 row affected (0.041 sec)

mysql>
mysql> INSERT INTO Staff VALUES (4, 104, 'Meena', 'Professor');
Query OK, 1 row affected (0.065 sec)
```

```
mysql> SELECT *
    -> FROM Department
    -> WHERE total_students >
    ->     (SELECT AVG(total_students) FROM Department);
+---------+-----------------------+------------+-------------+----------------+----------+------------------+
| dept_id | dept_name             | hod_name   | total_staff | total_students | building | established_year |
+---------+-----------------------+------------+-------------+----------------+----------+------------------+
|     101 | Computer Science      | Dr. Kumar  |          25 |            600 | Block A  |             2005 |
|     104 | Mechanical Engineering | Dr. Suresh |          30 |            700 | Block D  |             2000 |
+---------+-----------------------+------------+-------------+----------------+----------+------------------+
2 rows in set (0.097 sec)

mysql> SELECT *
    -> FROM Staff
    -> WHERE dept_id IN
    ->     (SELECT dept_id FROM Department
    ->      WHERE total_students > 500);
+----------+---------+------------+-------------+
| staff_id | dept_id | staff_name | designation |
+----------+---------+------------+-------------+
|        1 |     101 | Arun       | Professor   |
|        4 |     104 | Meena      | Professor   |
+----------+---------+------------+-------------+
2 rows in set (0.046 sec)
```

Subqueries with the INSERT Statement

- SQL subquery can also be used with the Insert statement. In the insert statement, datareturned from the subquery is used to insert into another table.
- In the subquery, the selected data can be modified with any of the character, datefunctions.

Syntax:

INSERT INTO table_name (column1, column2, column3 .....)
SELECT * FROM table_name WHERE VALUE OPERATOR

Subqueries with the UPDATE Statement

The subquery of SQL can be used in conjunction with the Update statement. When a subquery is used with the Update statement, then either single or multiple columns in a tablecan be updated.

Syntax:

UPDATE tableSET column_name = new_valueWHERE VALUE OPERATOR
(SELECT COLUMN_NAME FROMTABLE_NAME WHERE condition);

```
mysql> CREATE TABLE High_Student_Department (
    ->     dept_id INT,
    ->     dept_name VARCHAR(50),
    ->     total_students INT
    -> );
Query OK, 0 rows affected (0.410 sec)

mysql> INSERT INTO High_Student_Department (dept_id, dept_name, total_students)
    -> SELECT dept_id, dept_name, total_students
    -> FROM Department
    -> WHERE total_students >
    ->     (SELECT AVG(total_students) FROM Department);
Query OK, 2 rows affected (0.083 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> UPDATE Department
    -> SET total_students = total_students + (total_students * 0.05)
    -> WHERE building = 'Block A';
Query OK, 1 row affected (0.068 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Department;
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
| dept_id | dept_name                   | hod_name  | total_staff | total_students | building | established_year |
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
|     101 | Computer Science            | Dr. Kumar |          25 |           630 | Block A  |            2005 |
|     102 | Information Technology      | Dr. Priya |          20 |           450 | Block B  |            2008 |
|     103 | Electronics and Communication | Dr. Ravi  |          18 |           400 | Block C  |            2003 |
|     104 | Mechanical Engineering      | Dr. Suresh |          30 |           700 | Block D  |            2000 |
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
4 rows in set (0.012 sec)
```

Subqueries with the DELETE Statement

The subquery of SQL can be used in conjunction with the Delete statement just like any otherstatements mentioned above.

Syntax:

DELETE FROM TABLE_NAMEWHERE VALUE OPERATOR
(SELECT COLUMN_NAME FROM TABLE_NAME WHERE condition);

```
mysql> DELETE FROM Department
    -> WHERE building = 'Block A';
Query OK, 1 row affected (0.681 sec)

mysql> SELECT * FROM Department;
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
| dept_id | dept_name                   | hod_name  | total_staff | total_students | building | established_year |
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
|     102 | Information Technology      | Dr. Priya |          20 |           450 | Block B  |            2008 |
|     103 | Electronics and Communication | Dr. Ravi  |          18 |           400 | Block C  |            2003 |
|     104 | Mechanical Engineering      | Dr. Suresh |          30 |           700 | Block D  |            2000 |
+---------+-----------------------------+-----------+-------------+---------------+----------+-----------------+
3 rows in set (0.012 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

Ex.No:04	IMPLEMENTATION OF VIEWS AND JOINS

AIM:

DESCRIPTION:

- A view is a virtual table or logical representation of another table or combination of tables.
- Aview consists of rows and columns just like a table. The difference between a view and a table is thatviews are definitions built on top of other tables (or views), and do not hold data themselves.
- If data ischanging in the underlying table, the same change is reflected in the view.

ALGORITHM:

Step 1: Start.

Step 2: Go to MySQL.

Step 3: Enter the user name and password.Step

Step 4: Type the commands for creating, updating and dropping views

Step 5: The output is displayed.

Step 6: Stop.

1. CREATE VIEW:

Syntax:

CREATE VIEW view_nameAS SELECT column_list

FROM table_name [WHERE condition];

Description: To create a view from the table or another view.

```
mysql> CREATE VIEW Staff_View AS
    -> SELECT staff_id, staff_name, designation
    -> FROM Staff;
Query OK, 0 rows affected (0.106 sec)
```

2. TO VIEW THE VIEW

Syntax: select *from view_name;

Description: To view the view that is created.

```
mysql> SELECT * FROM Staff_View;
+----------+------------+--------------------+
| staff_id | staff_name | designation        |
+----------+------------+--------------------+
|        1 | Arun       | Professor          |
|        2 | Divya      | Assistant Professor |
|        3 | Karthik    | Lab Assistant      |
|        4 | Meena      | Professor          |
+----------+------------+--------------------+
4 rows in set (0.020 sec)
```

3. DROP VIEW:

Syntax: drop view view_name;

Description: To drop the view that is created.

```
mysql> DROP VIEW Staff_View;
Query OK, 0 rows affected (0.132 sec)
```

JOIN QUERIES:

Rules to perform join operation:

Before performing the join operations two tables should be connected using the primary and foreign key as following

1. INNER JOIN:

Syntax: select column_name(s) from table_name1 inner join table_name2 on table_name1.column_name=table_name2.column_name;

Description: To return rows when there is at least one match in both tables.

```
mysql> SELECT Staff.staff_id,
    ->        Department.dept_name,
    ->        Staff.staff_name
    -> FROM Staff
    -> INNER JOIN Department
    -> ON Staff.dept_id = Department.dept_id;
+----------+------------------------------+------------+
| staff_id | dept_name                    | staff_name |
+----------+------------------------------+------------+
|        2 | Information Technology       | Divya      |
|        3 | Electronics and Communication | Karthik    |
|        4 | Mechanical Engineering       | Meena      |
+----------+------------------------------+------------+
3 rows in set (0.018 sec)
```

2. LEFT JOIN:

Syntax: select column_name(s) from table_name1 left jointable_name2 on table_name1.column_name=table_name2.column_name;

Description: To return all rows from the left table (table_name1), even if there are nomatches in the right table (table_name2)

```
mysql> SELECT Staff.staff_id,
    ->        Staff.staff_name,
    ->        Department.dept_name
    -> FROM Staff
    -> LEFT JOIN Department
    -> ON Staff.dept_id = Department.dept_id;
+----------+------------+------------------------------+
| staff_id | staff_name | dept_name                    |
+----------+------------+------------------------------+
|        1 | Arun       | NULL                         |
|        2 | Divya      | Information Technology       |
|        3 | Karthik    | Electronics and Communication |
|        4 | Meena      | Mechanical Engineering       |
+----------+------------+------------------------------+
4 rows in set (0.017 sec)
```

3. RIGHT JOIN:

Syntax: select column_name(s)from table_name1 right join

table_name2 on table_name1.column_name=table_name2.column_name;

Description: To return all rows from the right table (table_name2), even if there are nomatches in the left table (table_name1).

```
mysql> SELECT Staff.staff_id,
    ->        Staff.staff_name,
    ->        Department.dept_name
    -> FROM Staff
    -> RIGHT JOIN Department
    -> ON Staff.dept_id = Department.dept_id;
+----------+------------+-----------------------------+
| staff_id | staff_name | dept_name                   |
+----------+------------+-----------------------------+
|        2 | Divya      | Information Technology      |
|        3 | Karthik    | Electronics and Communication |
|        4 | Meena      | Mechanical Engineering      |
+----------+------------+-----------------------------+
3 rows in set (0.013 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

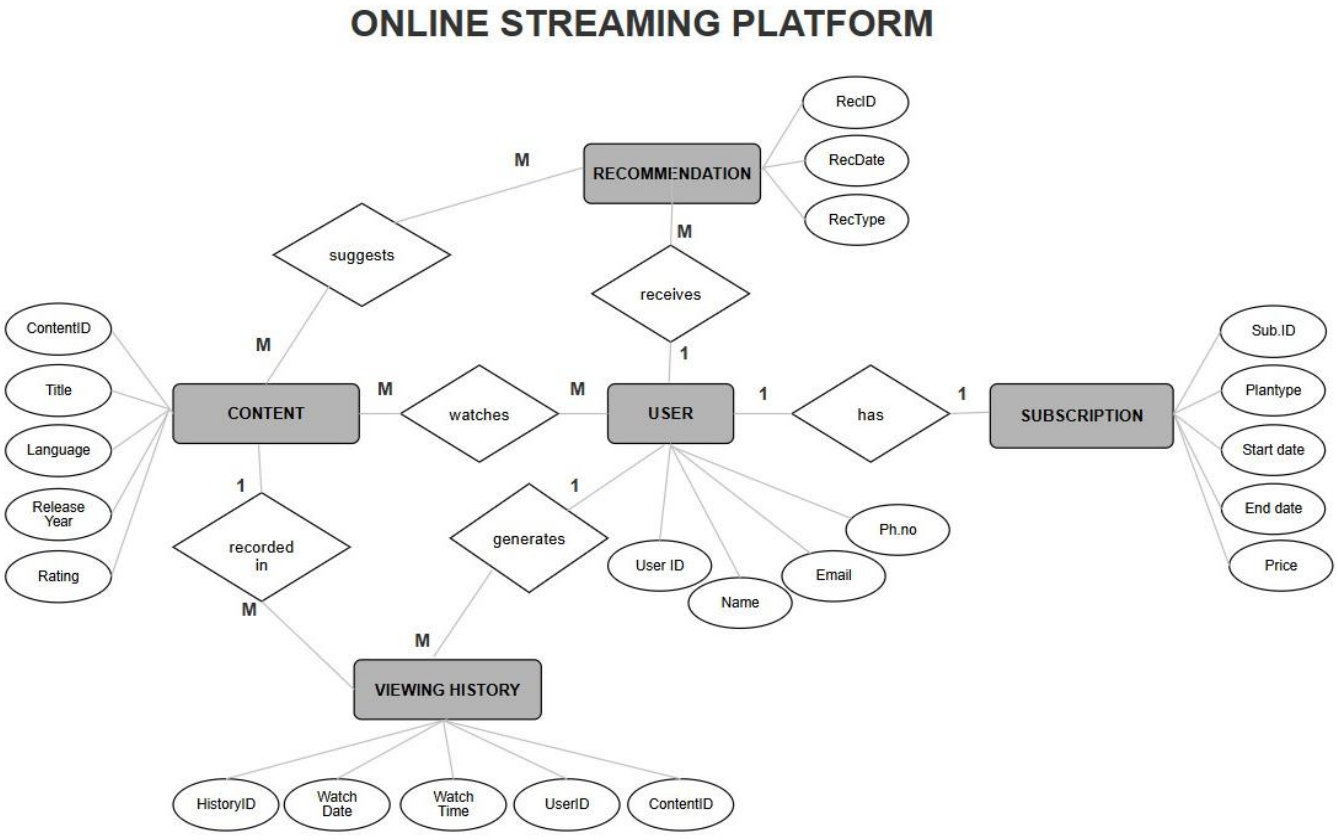
Signature of the Faculty

RESULT:

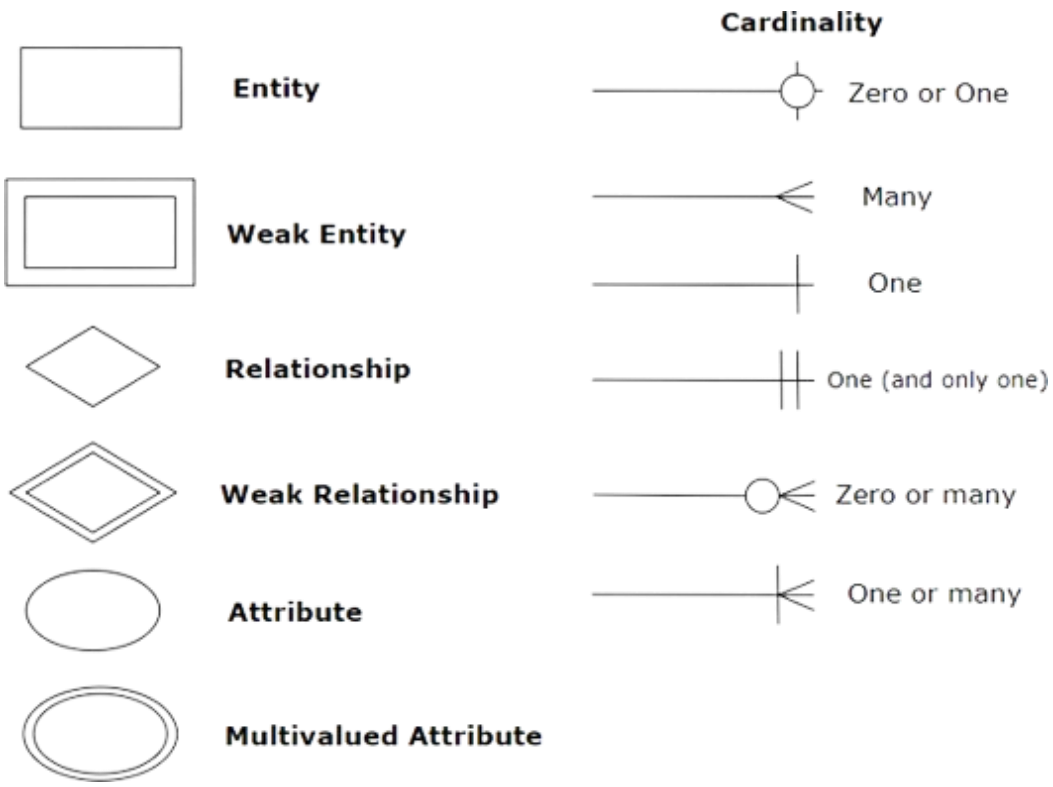
Ex.No:05	DATABASE DESIGN USING ER MODELING, NORMALIZATION AND IMPLEMENTATION FOR ANY APPLICATION

AIM:

ENTITY DIAGRAM:



NOTATION:


Entity
Weak Entity
Relationship
Weak Relationship
Attribute
Multivalued Attribute
Cardinality
Zero or One
Many
One
One (and only one)
Zero or many
One or many

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

Ex .No : 06	DATE, STRING AND NUMERIC FUNCTIONS

AIM:

ALGORITHM:

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the user name and password.
Step 4: Create database and table, and insert sample records.
Step 5: Type the commands to perform Date, String, and Numeric functions.
Step 6: Execute the queries and view the output.
Step 7: Stop.

Date Functions

- NOW() – Returns current date and time

```
mysql> CREATE TABLE Department (
    ->     dept_id INT,
    ->     dept_name VARCHAR(50),
    ->     total_students INT,
    ->     established_year DATE
    -> );
Query OK, 0 rows affected (0.344 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (101, 'Computer Science', 600, '2005-06-10'),
    -> (102, 'Information Technology', 450, '2008-07-15'),
    -> (103, 'Electronics', 400, '2003-05-20');
Query OK, 3 rows affected (0.120 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT NOW();
+---------------------+
| NOW()               |
+---------------------+
| 2026-05-11 22:27:24 |
+---------------------+
1 row in set (0.010 sec)
```

- CURDATE() / CURRENT_DATE – Current date

```
mysql> SELECT CURDATE(), CURRENT_DATE();
+------------+----------------+
| CURDATE()  | CURRENT_DATE() |
+------------+----------------+
| 2026-05-11 | 2026-05-11     |
+------------+----------------+
1 row in set (0.024 sec)
```

- CURTIME() – Current time

```
mysql> SELECT CURTIME();
+-----------+
| CURTIME() |
+-----------+
| 22:31:00  |
+-----------+
1 row in set (0.011 sec)
```

- DATE_ADD(date, INTERVAL n unit) – Adds time to date

```
mysql> SELECT dept_name,
    ->        established_year,
    ->        DATE_ADD(established_year, INTERVAL 10 DAY) AS New_Date
    -> FROM Department;
+-------------------------+-------------------+------------+
| dept_name               | established_year | New_Date   |
+-------------------------+-------------------+------------+
| Computer Science        | 2005-06-10       | 2005-06-20 |
| Information Technology | 2008-07-15       | 2008-07-25 |
| Electronics             | 2003-05-20       | 2003-05-30 |
+-------------------------+-------------------+------------+
3 rows in set (0.028 sec)
```

- DATEDIFF(date1, date2) – Difference between two dates

```
mysql> SELECT dept_name,
    ->       DATEDIFF(CURDATE(), established_year) AS Total_Days
    -> FROM Department;
+-----------------------+------------+
| dept_name             | Total_Days |
+-----------------------+------------+
| Computer Science      |       7640 |
| Information Technology |       6509 |
| Electronics           |       8392 |
+-----------------------+------------+
3 rows in set (0.015 sec)
```

- YEAR(date), MONTH(date), DAY(date) – Extract components

```
mysql> SELECT dept_name,
    ->        YEAR(established_year) AS Year,
    ->        MONTH(established_year) AS Month,
    ->        DAY(established_year) AS Day
    -> FROM Department;
+-----------------------+------+-------+------+
| dept_name             | Year | Month | Day  |
+-----------------------+------+-------+------+
| Computer Science      | 2005 |     6 |   10 |
| Information Technology | 2008 |     7 |   15 |
| Electronics           | 2003 |     5 |   20 |
+-----------------------+------+-------+------+
3 rows in set (0.035 sec)
```

String Functions:

```
mysql> SELECT dept_name,
    ->        LENGTH(dept_name) AS Length_Name
    -> FROM Department;
+-----------------------+-------------+
| dept_name             | Length_Name |
+-----------------------+-------------+
| Computer Science      |          16 |
| Information Technology |          22 |
| Electronics           |          11 |
+-----------------------+-------------+
3 rows in set (0.014 sec)
```

• LENGTH(str) – Returns length

```
mysql> SELECT CONCAT(dept_name, ' - Department') AS Full_Name
    -> FROM Department;
+----------------------------------+
| Full_Name                        |
+----------------------------------+
| Computer Science - Department    |
| Information Technology - Department |
| Electronics - Department         |
+----------------------------------+
3 rows in set (0.025 sec)
```

• UPPER(str) / LOWER(str) – Converts case

```
mysql> SELECT dept_name,
    ->        UPPER(dept_name) AS Upper_Name,
    ->        LOWER(dept_name) AS Lower_Name
    -> FROM Department;
+----------------------+----------------------+----------------------+
| dept_name            | Upper_Name           | Lower_Name           |
+----------------------+----------------------+----------------------+
| Computer Science     | COMPUTER SCIENCE     | computer science     |
| Information Technology | INFORMATION TECHNOLOGY | information technology |
| Electronics          | ELECTRONICS          | electronics          |
+----------------------+----------------------+----------------------+
3 rows in set (0.038 sec)
```

• SUBSTRING(str, start, length) – Extracts substring

```
mysql> SELECT dept_name,
    ->        SUBSTRING(dept_name, 1, 3) AS Short_Name
    -> FROM Department;
+------------------------+------------+
| dept_name              | Short_Name |
+------------------------+------------+
| Computer Science       | Com        |
| Information Technology | Inf        |
| Electronics            | Ele        |
+------------------------+------------+
3 rows in set (0.014 sec)
```

• REPLACE(str, from_str, to_str) – Replaces part of string

```
mysql> SELECT dept_name,
    ->        REPLACE(dept_name, 'a', '@') AS Modified_Name
    -> FROM Department;
+----------------------+----------------------+
| dept_name            | Modified_Name        |
+----------------------+----------------------+
| Computer Science     | Computer Science     |
| Information Technology | Inform@tion Technology |
| Electronics          | Electronics          |
+----------------------+----------------------+
3 rows in set (0.012 sec)
```

• TRIM(str) – Removes whitespace.

```
mysql> SELECT TRIM('   Computer Science   ') AS Trimmed_Text;
+-----------------+
| Trimmed_Text    |
+-----------------+
| Computer Science |
+-----------------+
1 row in set (0.007 sec)
```

Numeric Functions:

• ROUND(number, decimals) – Rounds number

```
mysql> SELECT total_students,
    ->        ROUND(total_students, 0) AS Rounded_Value
    -> FROM Department;
+----------------+---------------+
| total_students | Rounded_Value |
+----------------+---------------+
|            600 |           600 |
|            450 |           450 |
|            400 |           400 |
+----------------+---------------+
3 rows in set (0.015 sec)
```

• FLOOR(number) / CEIL(number) – Floor and ceiling

```
mysql> SELECT total_students,
    ->        FLOOR(total_students) AS Floor_Value,
    ->        CEIL(total_students) AS Ceil_Value
    -> FROM Department;
+----------------+-------------+------------+
| total_students | Floor_Value | Ceil_Value |
+----------------+-------------+------------+
|            600 |         600 |        600 |
|            450 |         450 |        450 |
|            400 |         400 |        400 |
+----------------+-------------+------------+
3 rows in set (0.014 sec)
```

ABS(number) – Absolute value

```
mysql> SELECT ABS(-500) AS Absolute_Value;
+----------------+
| Absolute_Value |
+----------------+
|            500 |
+----------------+
1 row in set (0.026 sec)
```

• POWER(x, y) – x raised to power y

```
mysql> SELECT POWER(2, 3) AS Power_Value;
+-------------+
| Power_Value |
+-------------+
|           8 |
+-------------+
1 row in set (0.021 sec)
```

• RAND() – Random number

```
mysql> SELECT RAND() AS Random_Value;
+--------------------+
| Random_Value       |
+--------------------+
| 0.34406850705302183 |
+--------------------+
1 row in set (0.034 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

Ex. No:07	DATABASE PROGRAMMING: IMPLICIT AND EXPLICIT CURSORS

AIM:

ALGORITHM:

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the user name and password.
Step 4: Create database and table.
Step 5: Insert sample records into the table.
Step 6: Use UPDATE statement (implicit cursor).
Step 7: Declare cursor in stored procedure.
Step 8: Open the cursor.
Step 9: Fetch records using loop.
Step 10: Display the data.
Step 11: Close the cursor.
Step 12: Execute the procedure and view the output.
Step 13: Stop.

DESCRIPTION

PL/SQL stands for Procedural Language extension of SQL.PL/SQL is a combination of SQL along with the procedural features of programming languages.

Each PL/SQL program consists of SQL and PL/SQL statements which from a PL/SQLblock. APL/SQL Block consists of three sections:

- The Declaration section (optional).
- The Execution section (mandatory).
- The Exception or Error Handling section (optional).

Declaration Section: The Declaration section of a PL/SQL Block starts with the reserved keyword DECLARE. This section is optional and is used to declare any placeholders like variables, constants, records and cursors, which are used to manipulate data in the execution section. Placeholders may be any of Variables, Constants and Records, which stores datatemporarily. Cursors are also declared in this section.

Execution Section: The Execution section of a PL/SQL Block starts with the reserved keyword BEGIN and ends with END. This is a mandatory section and is the section where the program logic is written to perform any task. The programmatic constructs like loops, conditional statement and SQL statementsform the part of execution section.

Exception Section: The Exception section of a PL/SQL Block starts with the reserved keyword EXCEPTION. This section is optional. Any errors in the program can be handled in this section.

DECLARE

Variable declaration

BEGIN

Program Execution

EXCEPTION

Exception handling

END;

DECLARING A CURSOR

A cursor declaration defines the SELECT statement for the query you need to process in the body ofthe block. In its simple form, a cursor declaration has the following syntax:

CURSOR cursor_name IS select_statement;

The cursor name is an undeclared PL/SQL variable. You cannot assign values to a cursor name or use it in an expression. The following example opens a cursor on the EMP table for records with SALARY above 2000. If there is no WHERE clause, all rows from the EMP table will be processed.

```
DECLARE
CURSOR C_EMP IS SELECT empno, ename, salaryFROM emp WHERE
salary > 2500ORDER BY ename;
```

OPENING A CURSOR

The cursor should be opened before its row values can be used. Opening the cursor initiates the query processing. Here is the syntax for opening a simple cursor:

OPEN cursor_name;

Where cursor name is the name of the cursor defined in the declaration section. For example, to open the cursor defined on EMP table, do this:

OPEN C_EMP;

FETCHING A CURSOR

A cursor is fetched to retrieve a row at a time; use the FETCH command for this. After each fetch, the cursor advances to the next row in the result set. Here is the syntax of the FETCHcommand:

FETCH cursor_name INTO variable [, variable]…;

For each column in the cursor definition SELECT, there should be a corresponding variable in the FETCH variable list. The variables should be declared with the appropriate data type in the declarationsection.

CLOSING A CURSOR

Explicitly opened cursors should be closed explicitly. The syntax for closing a cursor follows:

CLOSE cursor_name;

For example, to close the cursor defined on the EMP table, do this:

CLOSE C_EMP;

```
mysql> CREATE DATABASE Dept_Demo;
Query OK, 1 row affected (0.103 sec)

mysql> USE Dept_Demo;
Database changed
mysql> CREATE TABLE Department (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50),
    ->     total_students INT
    -> );
Query OK, 0 rows affected (0.350 sec)

mysql> INSERT INTO Department VALUES
    -> (101, 'Computer Science', 600),
    -> (102, 'Information Technology', 450),
    -> (103, 'Electronics', 400),
    -> (104, 'Mechanical', 700);
Query OK, 4 rows affected (0.085 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> UPDATE Department
    -> SET total_students = total_students + 100
    -> WHERE total_students > 500;
Query OK, 2 rows affected (0.078 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> SELECT ROW_COUNT() AS Rows_Affected;
+---------------+
| Rows_Affected |
+---------------+
|             2 |
+---------------+
1 row in set (0.020 sec)
```

```
mysql>
mysql> CREATE PROCEDURE dept_cursor_example()
    -> BEGIN
    ->     DECLARE done INT DEFAULT 0;
    ->     DECLARE v_id INT;
    ->     DECLARE v_name VARCHAR(50);
    ->     DECLARE v_students INT;
    ->
    ->     DECLARE dept_cursor CURSOR FOR
    ->         SELECT dept_id, dept_name, total_students
    ->         FROM Department
    ->         WHERE total_students > 500;
    ->
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->     OPEN dept_cursor;
    ->
    ->     read_loop: LOOP
    ->
    ->         FETCH dept_cursor INTO v_id, v_name, v_students;
    ->
    ->         IF done = 1 THEN
    ->             LEAVE read_loop;
    ->         END IF;
    ->
    ->         SELECT v_id AS ID,
    ->                v_name AS Department_Name,
    ->                v_students AS Total_Students;
    ->
    ->     END LOOP;
    ->
    ->     CLOSE dept_cursor;
    ->
    -> END $$
Query OK, 0 rows affected (0.146 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL dept_cursor_example();
+------+------------------+----------------+
| ID   | Department_Name  | Total_Students |
+------+------------------+----------------+
|  101 | Computer Science |            700 |
+------+------------------+----------------+
1 row in set (0.048 sec)

+------+-----------------+----------------+
| ID   | Department_Name | Total_Students |
+------+-----------------+----------------+
|  104 | Mechanical      |            800 |
+------+-----------------+----------------+
1 row in set (0.186 sec)

Query OK, 0 rows affected (0.310 sec)
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

EX NO:8	PROCEDURES AND FUNCTIONS

AIM

ALGORITHM:

(Procedure: get_merit_student)

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the user name and password.
Step 4: Create table `student_info` with required fields.
Step 5: Insert sample records into the table.
Step 6: Create procedure `get_merit_student`.
Step 7: Inside the procedure, display students whose marks are greater than 70.
Step 8: Count total number of students.
Step 9: Call the procedure.
Step 10: Stop.

(Function: count_merit_students)

Step 1: Start.
Step 2: Go to MySQL.
Step 3: Enter the user name and password.
Step 4: Create function `count_merit_students`.
Step 5: Declare a variable to store count.
Step 6: Count students whose marks are greater than 70.
Step 7: Return the count value.
Step 8: Call the function using `SELECT`.
Step 9: Stop.

PROCEDURE

General Syntax to create a procedure is

CREATE [OR REPLACE] PROCEDURE proc_name [list of parameters]

IS

Declaration section

BEGIN

Execution section

EXCEPTION

Exception section

END;

The below example creates a procedure 'get_merit_student' which gives the details of the merit students.Create the student-info table as follows:

```
CREATE PROCEDURE get_merit_student ()
    BEGIN
        SELECT * FROM student_info WHERE marks > 70;
        SELECT COUNT(stud_code) AS Total_Student FROM student_info;
 END &&
DELIMITER ;
```

Call the Procedure:

```
mysql> CALL get_merit_student();
```

```
mysql> CREATE TABLE student_info (
    ->     stud_code INT PRIMARY KEY,
    ->     stud_name VARCHAR(50),
    ->     marks INT
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO student_info VALUES (1, 'Arun', 85);
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO student_info VALUES (2, 'Bala', 65);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student_info VALUES (3, 'Cathy', 90);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student_info VALUES (4, 'David', 72);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student_info VALUES (5, 'Eva', 60);
Query OK, 1 row affected (0.02 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE get_merit_student()
    -> BEGIN
    ->     -- Display students with marks > 70
    ->     SELECT * FROM student_info WHERE marks > 70;
    ->
    ->     -- Count total students
    ->     SELECT COUNT(stud_code) AS Total_Students FROM student_info;
    -> END $$
Query OK, 0 rows affected (0.03 sec)
```

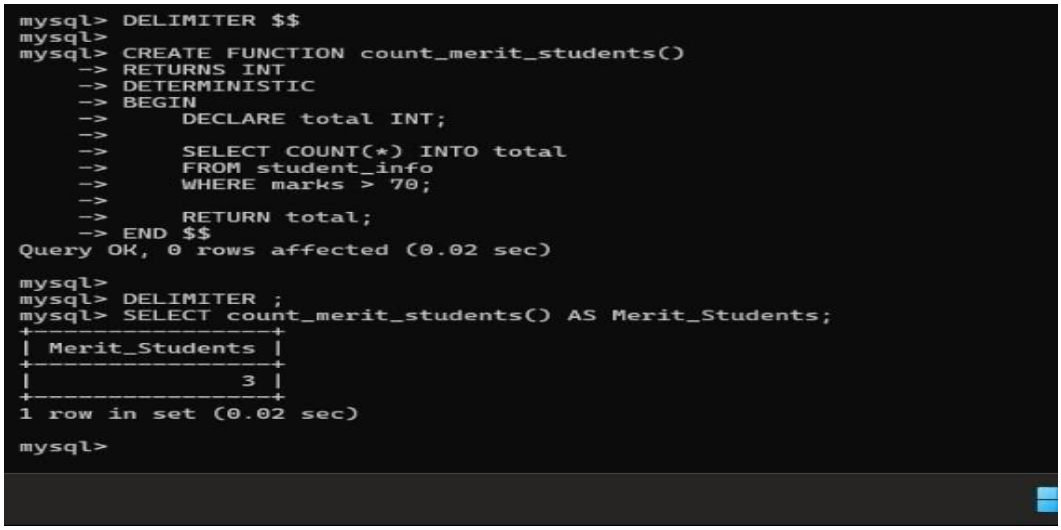
```
mysql>
mysql> DELIMITER ;
mysql> CALL get_merit_student();
+------------+------------+--------+
| stud_code  | stud_name  | marks  |
+------------+------------+--------+
|          1 | Arun       |     85 |
|          3 | Cathy      |     90 |
|          4 | David      |     72 |
+------------+------------+--------+
3 rows in set (0.00 sec)

+------------------+
| Total_Students   |
+------------------+
|                5 |
+------------------+
1 row in set (0.03 sec)
```

FUNCTION

The General Syntax to create a function is:

CREATE [OR REPLACE] FUNCTION function_name [parameters]
RETURN return_datatype;**IS**
Declaration_section **BEGIN**
Execution_section
Return return_variable;
EXCEPTION exception section Return return_variable;
END;



```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION count_merit_students()
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->
    ->     SELECT COUNT(*) INTO total
    ->     FROM student_info
    ->     WHERE marks > 70;
    ->
    ->     RETURN total;
    -> END $$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT count_merit_students() AS Merit_Students;
+-----------------+
| Merit_Students |
+-----------------+
|               3 |
+-----------------+
1 row in set (0.02 sec)

mysql>
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

EX NO:9	EMBEDDED SQL

AIM

ALGORITHM:

Step 1: Start.
Step 2: Create Database.
Step 3: Select database.
Step 4 Create Table EMPOLYEE.
Step 5: Insert sample records into the table.
Step 6: Stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <mysql/mysql.h>
MYSQL *conn;
MYSQL_RES *res;
MYSQL_ROW row;
void connectDB() {
   conn = mysql_init(NULL);
   if (!mysql_real_connect(conn,
                    "localhost",
                    "root",
                    "Vennimalaimohan@006",
                    "company",
                    0,
                    NULL,
                    0)) {

      printf("Database Connection Failed\n");
```

```
        exit(1);
    }
    printf("\nDatabase Connected Successfully\n");
}
void insertCustomer() {
    int id;
    char name[50];
    char city[50];
    char query[200];
    printf("\nEnter Customer ID : ");
    scanf("%d", &id);
    printf("Enter Customer Name : ");
    scanf("%s", name);
    printf("Enter Customer City : ");
    scanf("%s", city);
    sprintf(query,
        "INSERT INTO customer VALUES(%d, '%s', '%s')",
        id, name, city);
    if (mysql_query(conn, query)) {
        printf("Insert Failed\n");
    }
    else {
        printf("Customer Inserted Successfully\n");
    }
}
void displayCustomers() {
    if (mysql_query(conn, "SELECT * FROM customer")) {

        printf("Display Failed\n");
        return;
    }
```

```
    res = mysql_store_result(conn);
    printf("\n--- Customer Records ---\n");
    while ((row = mysql_fetch_row(res))) {
        printf("ID    : %s\n", row[0]);
        printf("Name  : %s\n", row[1]);
        printf("City  : %s\n", row[2]);

        printf("----------------\n");
    }
    mysql_free_result(res);
}
void updateCustomer() {
    int id;
    char city[50];
    char query[200];
    printf("\nEnter Customer ID to Update : ");
    scanf("%d", &id);
    printf("Enter New City : ");
    scanf("%s", city);
    sprintf(query,
        "UPDATE customer SET city='%s' WHERE id=%d",
        city, id);
    if (mysql_query(conn, query)) {
        printf("Update Failed\n");
    }
    else {
        printf("Customer Updated Successfully\n");
    }
}
```

```
void deleteCustomer() {
    int id;
    char query[200];
    printf("\nEnter Customer ID to Delete : ");
    scanf("%d", &id);
    sprintf(query,
        "DELETE FROM customer WHERE id=%d",
        id);
    if (mysql_query(conn, query)) {
        printf("Delete Failed\n");
    }
    else {
        printf("Customer Deleted Successfully\n");
    }
}
int main() {
    int choice;
    connectDB();
    while (1) {
        printf("\n====== CUSTOMER MENU ======\n");
        printf("1. Insert Customer\n");
        printf("2. Display Customers\n");
        printf("3. Update Customer\n");
        printf("4. Delete Customer\n");
        printf("5. Exit\n");
        printf("Enter Choice : ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                insertCustomer();
                break;
```

```
            case 2:
                displayCustomers();
                break;
            case 3:
                updateCustomer();
                break;
            case 4:
                deleteCustomer();
                break;
            case 5:
                mysql_close(conn);
                printf("Program Exited\n");
                exit(0);
            default:
                printf("Invalid Choice\n");
        }
    }
    return 0;
}
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2

vennimalaimohans@VENNIMALAIMOHANs-MacBook-Air Practise % ./customer
Database Connected Successfully

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 1

Enter Customer ID : Mohan S
Enter Customer Name : Enter Customer City : Customer Inserted Successfully

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 2

--- Customer Records ---
ID    : 0
Name  : Mohan
City  : S
--------------------------
ID    : 1
Name  : Mohan
City  : Nidamangalam
--------------------------

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 4

Enter Customer ID to Delete : 0
Customer Deleted Successfully

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 2
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2

vennimalaimohans@VENNIMALAIMOHANs-MacBook-Air Practise % ./customer
Name  : Mohan
City  : Nidamangalam
--------------------------

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 4

Enter Customer ID to Delete : 0
Customer Deleted Successfully

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 2

--- Customer Records ---
ID    : 1
Name  : Mohan
City  : Nidamangalam
--------------------------

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : 3

Enter Customer ID to Update : 1
Enter New City : Thiruchengode
Customer Updated Successfully

====== CUSTOMER MENU ======
1. Insert Customer
2. Display Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter Choice : ^C
vennimalaimohans@VENNIMALAIMOHANs-MacBook-Air Practise %
```

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT:

EX NO:10	MYSQL SIMPLE QUERIES AND DATABASE CONNECTIVITY

AIM

ALGORITHM:

Step 1: Start.
Step 2: Import required Java SQL and Scanner packages.
Step 3: Define database URL, username, and password.
Step 4: Load MySQL JDBC driver.
Step 5: Establish connection with MySQL database.
Step 6: Create Scanner object for user input.
Step 7: Enter Employee ID.
Step 8: Enter Employee Name.
Step 9: Enter Department.
Step 10: Enter Salary.
Step 11: Create SQL INSERT query using `PreparedStatement`.
Step 12: Set employee values in the query.
Step 13: Execute the INSERT query.
Step 14: Display success message after inserting record.
Step 15: Close database connection and Scanner object.
Step 16: Stop.

PROGRAM:

```
#include <mysql/mysql.h>

#include <stdio.h>

#include <stdlib.h>

MYSQL *conn;

MYSQL_RES *res;

MYSQL_ROW row;

void connectDatabase() {

    conn = mysql_init(NULL);
```

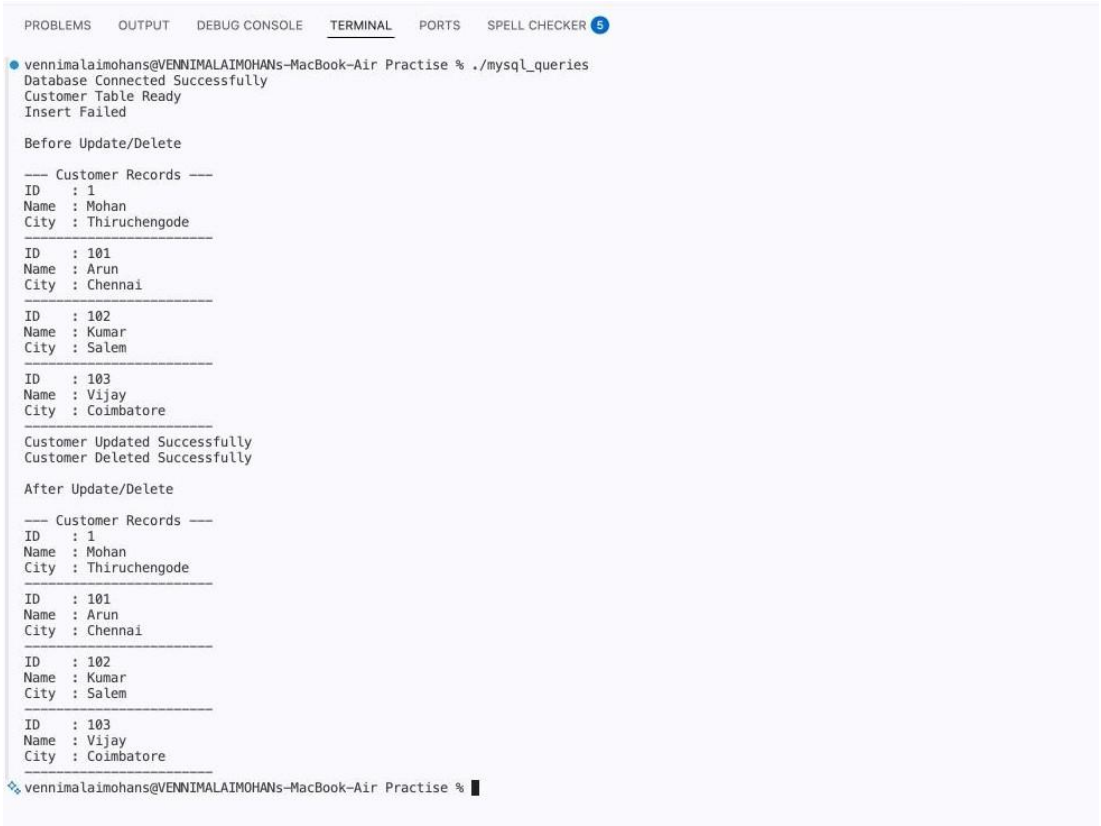
```
    if (!mysql_real_connect(conn,
                            "localhost",
                            "root",
                            "Vennimalaimohan@006",
                            "company",
                            0,
                            NULL,
                            0)) {
        printf("Database Connection Failed\n");
        exit(1);
    }
    printf("Database Connected Successfully\n");
}
void createTable() {
    char query[] =
    "CREATE TABLE IF NOT EXISTS customer ("
    "id INT PRIMARY KEY,"
    "name VARCHAR(50),"
    "city VARCHAR(50))";
    if (mysql_query(conn, query)) {
        printf("Table Creation Failed\n");
    }
```

```
    else {
        printf("Customer Table Ready\n");
    }
}
void insertCustomer() {
    char query[] =
    "INSERT INTO customer VALUES "
    "(101,'Arun','Chennai'),"
    "(102,'Kumar','Salem'),"
    "(103,'Vijay','Coimbatore')";
    if (mysql_query(conn, query)) {
        printf("Insert Failed\n");
    }
    else {
        printf("Records Inserted Successfully\n");
    }
}
void displayCustomers() {
    if (mysql_query(conn,
        "SELECT * FROM customer")) {
        printf("Select Query Failed\n");
        return;
    }
```

```
    res = mysql_store_result(conn);
    printf("\n--- Customer Records ---\n");
    while ((row = mysql_fetch_row(res))) {
        printf("ID    : %s\n", row[0]);
        printf("Name  : %s\n", row[1]);
        printf("City  : %s\n", row[2]);
        printf("----------------\n");
    }
    mysql_free_result(res);
}
void updateCustomer() {
    char query[] =
    "UPDATE customer SET city='Madurai' "
    "WHERE id=2";
    if (mysql_query(conn, query)) {
        printf("Update Failed\n");
    }
    else {
        printf("Customer Updated Successfully\n");
    }
}
```

```
void deleteCustomer() {
    char query[] =
    "DELETE FROM customer WHERE id=3";
    if (mysql_query(conn, query)) {
        printf("Delete Failed\n");
    }
    else {
        printf("Customer Deleted Successfully\n");
    }
}
int main() {
    connectDatabase();
    createTable();
    insertCustomer();
    printf("\nBefore Update/Delete\n");
    displayCustomers();
    updateCustomer();
    deleteCustomer();
    printf("\nAfter Update/Delete\n");
    displayCustomers();
    mysql_close(conn);
    return 0;
}
```

OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER 5
vennimalaimohans@VENNIMALAIMOHANs-MacBook-Air Practise % ./mysql_queries
Database Connected Successfully
Customer Table Ready
Insert Failed
Before Update/Delete
--- Customer Records ---
ID : 1
Name : Mohan
City : Thiruchengode
ID : 101
Name : Arun
City : Chennai
ID : 102
Name : Kumar
City : Salem
ID : 103
Name : Vijay
City : Coimbatore
Customer Updated Successfully
Customer Deleted Successfully
After Update/Delete
--- Customer Records ---
ID : 1
Name : Mohan
City : Thiruchengode
ID : 101
Name : Arun
City : Chennai
ID : 102
Name : Kumar
City : Salem
ID : 103
Name : Vijay
City : Coimbatore
vennimalaimohans@VENNIMALAIMOHANs-MacBook-Air Practise %

DETAILS	MARKS ALLOCATED	MARKS AWARDED
Program	40	
Error Free Compilation	30	
Result	20	
Viva-Voce	10	
Total	100	

Signature of the Faculty

RESULT: